

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Penjelasan :

- Ini adalah kelas dasar untuk merepresentasikan satu elemen pada linked list.
- `data` menyimpan nilai dari node tersebut.
- `next` menunjuk ke node berikutnya dalam stack (linked list).

```
class StackLinkedList:
    def __init__(self, capacity):
        self.top = None
        self.capacity = capacity
        self.count = 0
```

Penjelasan:

- Membuat objek stack berbasis linked list.
- `top` menunjuk ke node paling atas (terbaru ditambahkan).
- `capacity` adalah batas maksimum elemen stack.
- `count` adalah jumlah elemen saat ini di stack.

```
def push(self, data):
    if self.is_full():
        print("Stack sudah penuh!")
        return
    new_node = Node(data)
    new_node.next = self.top
    self.top = new_node
    self.count += 1
```

Penjelasan:

- Menambahkan elemen baru ke stack.
- Jika stack sudah penuh, tampilkan pesan dan hentikan.
- Jika belum penuh, buat node baru dan letakkan di paling atas stack.
- Update penunjuk `top` dan tambahkan jumlah elemen (`count`).

```
def pop(self):
    if self.top is None:
        print("Stack kosong, tidak bisa menghapus!")
        return
    removed = self.top.data
    self.top = self.top.next
    self.count -= 1
    print(f"Elemen {removed} telah dihapus.")
```

Penjelasan:

- Menghapus elemen teratas dari stack.
- Jika stack kosong, tampilkan pesan.
- Jika tidak, ambil nilai `top`, update `top` ke node berikutnya, dan kurangi `count`.

```
def size(self):
    print(f"Ukuran stack saat ini: {self.count}")
```

Penjelasan:

- Menampilkan jumlah elemen yang saat ini ada di dalam stack

```
def peek(self):
    if self.top is None:
        print("Stack kosong.")
    else:
        print(f"Puncak stack saat ini: {self.top.data}")
```

Penjelasan:

- Menampilkan elemen paling atas tanpa menghapusnya.
- Cek apakah stack kosong terlebih dahulu.

```
def is_full(self):
    return self.count == self.capacity
```

Penjelasan:

- Mengecek apakah stack sudah mencapai kapasitas maksimal.

```
def display(self):
    current = self.top
    items = []
    while current:
        items.append(current.data)
        current = current.next
    print("Stack:", items[::-1]) # Ditampilkan dari bawah ke atas
```

Penjelasan:

- Menampilkan seluruh isi stack dari **bawah ke atas**.
- Gunakan list `items` untuk menampung data dan balikkan urutannya untuk mencetak dari dasar ke puncak stack.

```
# ===== PROGRAM UTAMA =====
def main():
    print("=====PROGRAM SEDERHANA UNTUK IMPLEMENTASI STACK DENGAN LINKED-  
LIST=====")
    kapasitas = int(input("Tentukan berapa kapasitas stack : "))
    stack = StackLinkedList(kapasitas)

    def display(self):
        current = self.top
        items = []
        while current:
            items.append(current.data)
            current = current.next
        print("Stack:", items[::-1]) # Ditampilkan dari bawah ke atas
```

Penjelasan:

- Ini adalah fungsi utama program tempat interaksi dengan pengguna terjadi.
- Program meminta pengguna menentukan kapasitas stack, lalu masuk ke dalam **loop menu**.

```

while True:

    print("\nPilih menu berikut ini :")
    print("1. Menambah isi stack")
    print("2. Menghapus isi stack")
    print("3. Cek Ukuran Stack saat ini")
    print("4. Cek Puncak Stack")
    print("5. Cek Stack Full")
    print("6. Keluar")

    pilihan = input("Masukkan pilihan anda : ")

    if pilihan == '1':
        while not stack.is_full():
            data = input("Masukkan isi stack : ")
            stack.push(data)
            stack.display()
            if stack.is_full():
                print("Stack sudah penuh.")
                break
            lanjut = input("Menambah isi Stack Pilih [Ya/Tidak] : ").lower()
            if lanjut != 'ya':
                break
    elif pilihan == '2':
        stack.pop()
        stack.display()
    elif pilihan == '3':
        stack.size()
    elif pilihan == '4':
        stack.peak()
    elif pilihan == '5':
        if stack.is_full():
            print("Stack dalam kondisi penuh.")
        else:
            print("Stack masih tersedia.")
    elif pilihan == '6':
        print("Terima kasih telah menggunakan program.")
        break
    else:
        print("Pilihan tidak valid. Coba lagi.")

```

Penjelasan:

- Memberikan opsi kepada pengguna:
 1. Tambah elemen ke stack.
 2. Hapus elemen dari stack.

3. Tampilkan ukuran saat ini.
4. Lihat elemen teratas.
5. Cek apakah stack penuh.
6. Keluar dari program.

Khusus menu 1 (push):

- Akan terus menambahkan elemen sampai stack penuh atau pengguna memilih berhenti.

```
if __name__ == "__main__":  
    main()
```

Penjelasan:

- Menjalankan fungsi `main()` hanya jika file ini dijalankan secara langsung, bukan diimpor sebagai modul.